

STAGE-VERIFIABLE LATENT CHAIN-OF-THOUGHT WITH LOCAL REPAIR FOR LONG-HORIZON ROBOTIC MANIPULATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Long-horizon robotic manipulation requires reasoning to remain consistent with execution, yet continuous latent plans often lack observable stage semantics. We propose a vision-language-action framework that aligns latent segments with demonstration-derived object relations and subgoal completion, verifies them against observations, and repairs affected reasoning after execution deviations. The model retains a verified prefix whose prerequisites remain valid and regenerates the affected suffix from current observations and task history. All model calls, including verification and repair, contribute to the task-level inference budget. The method achieves measured mean success rates of 98.0% overall and 96.8% on LIBERO-Long, and 58.0%/23.0% under clean/randomized conditions across 50 RoboTwin 2.0 tasks. On LIBERO-RECOVER, enabling repair improves the recovery macro-average from 22.8% to 26.0%. Compared with full replanning, local repair reduces cumulative inference time from 18.6 to 14.9 seconds per episode, a 19.9% reduction, with recovery macro-averages of 26.0% and 26.4%, respectively. These measurements support a trade-off between recovery and computation under the evaluated settings.

1 INTRODUCTION

Long-horizon robotic manipulation requires more than producing a plausible sequence of actions. Opening a drawer, retrieving an object, and placing it in a container involves subgoals whose outcomes become prerequisites for later decisions. A missed grasp or displaced object can break these dependencies even when the task instruction remains unchanged. If subsequent actions continue to rely on the expected outcome, a local execution error can propagate through the remaining task. Maintaining consistency between intermediate reasoning and observed progress is therefore central to reliable continuation.

This consistency problem has both a representational and a computational aspect. A robot needs to distinguish an intended state transition from one supported by execution evidence. It also needs to determine how much of its reasoning to revisit after the two diverge. Reusing an entire plan can preserve invalid assumptions, whereas rebuilding all remaining representations after each deviation can discard useful intermediate computation. The useful unit of reuse is therefore task progress whose consequences remain valid for the operations that follow. Physical actions may change the scene even when the language goal stays fixed, making this distinction necessary throughout an episode.

Vision-language-action (VLA) models can use intermediate representations to connect task understanding with control. Continuous latent reasoning provides a compact way to organize such information without generating a full textual plan at every decision. Recent approaches develop latent planning, physical-state supervision, multimodal alignment, and adaptive reasoning depth (Huang et al., 2026; Liu et al., 2026b; Zhong et al., 2026; Lei & Shan, 2026). However, a representation that predicts a plausible future state is not itself evidence that the corresponding subgoal has been achieved. For closed-loop execution, the model also needs an observable interpretation of its intermediate stages and a rule for updating the reasoning affected by a deviation.

Execution monitors and task-graph systems address complementary aspects of this problem by diagnosing failures, preparing recovery branches, and revising plans (Ying et al., 2026; Xu et al., 2026;

Gu et al., 2026). Progress reuse and local recovery are thus established ideas. We focus on a specific connection inside a VLA model: using observation-based stage verification to decide which continuous latent segments remain usable and which must be regenerated. This requires representation learning, progress checking, and repair-scope selection to operate on a common stage semantics.

Object relations and subgoal events provide such semantics in the demonstration data. Relations such as an object being held or entering a target container describe changes that can be checked against observations. The corresponding completion signals connect those changes to the instruction. Aligning a latent segment with these events makes its expected outcome explicit without requiring the full reasoning process to be rendered as text. Verification can then compare what the representation anticipates with what execution supports, rather than interpreting the plausibility of a predicted state as evidence of success.

We propose a framework that aligns latent segments with demonstration-derived object relations and subgoal completion. A latent-state decoder predicts each stage’s expected outcome, while an observation-based estimator supplies evidence about execution. Supported completion advances the plan; reliable, persistent deviations trigger regeneration from the earliest affected stage. The model retains a verified prefix only while its still-required conditions remain valid. For example, a failed grasp need not undo an already opened drawer, whereas a changed prerequisite can require repair from an earlier stage. The new suffix is conditioned on the current scene and valid task history, allowing subsequent actions to reflect what actually happened.

The two components have complementary roles in this loop. Stage alignment makes an expected result available for checking, but cannot by itself change a plan after an unsuccessful action. Feedback-driven repair uses that check to revise the representations conditioning future control, but depends on a meaningful correspondence between latent stages and observations. Together they make the scope of recomputation responsive to execution. The potential saving comes from retaining valid intermediate context; whether it outweighs monitoring and reconstruction costs is an empirical question, addressed by the repair-strategy comparison.

Evaluation must examine this feedback loop beyond success from ordinary initial states. We use LIBERO for language-conditioned manipulation, RoboTwin 2.0 for clean and randomized dual-arm tasks, and LIBERO-RECOVER for continuation from failure states. Controlled ablations separate additional state supervision, alignment of the latent chain, and the use of verification for repair. A complementary comparison varies the reconstruction scope while accounting for the model calls needed to monitor execution and generate updated actions. This connects the proposed mechanism to both its recovery benefits and its computational cost.

Our contributions are threefold: a stage-supervised latent representation with an observation-based verification interface; a feedback-driven suffix update that connects verified progress to the scope of latent regeneration; and an evaluation of this design’s task performance, recovery, and computation. All model calls, including verification and repair, enter a task-level budget. On LIBERO, the method reaches 98.0% average success and 96.8% on Long tasks. Controlled repair comparisons show a 19.9% reduction in complete-episode inference time relative to full replanning, with recovery macro-averages of 26.0% and 26.4%, respectively. These results motivate examining recovery and cost jointly rather than treating either as a sufficient measure of the framework.

2 RELATED WORK

Reasoning representations for manipulation. Structured intermediate representations connect high-level intent to executable behavior. Embodied-R1 uses spatial pointing as an embodiment-agnostic interface, Action-Sketcher expresses spatial intentions through editable visual sketches, and TCoT combines global and local trajectory reasoning with failure recovery (Yuan et al., 2026; Tan et al., 2026; Li et al., 2026). These methods show that intermediate reasoning can support both action selection and execution adjustment. Our representation is a sequence of continuous latent segments. Object relations and subgoal signals give these segments observable stage meanings, so the same representation can condition actions and identify where expected progress disagrees with execution.

This distinction concerns the interface between representation and feedback: a spatial intention specifies what the robot should do, while a stage outcome also supplies a condition against which the result can be checked. The two roles can coexist within a manipulation system.

Latent planning and adaptive computation. Fast-ThinkAct distills language and visual planning into compact latent representations, while LaRA-VLA transitions from explicit text-visual supervision to latent reasoning for action generation (Huang et al., 2026; Bai et al., 2026). LaST₀ introduces physical information through spatiotemporal latent modeling, and DualCoT-VLA combines visual and linguistic supervision through parallel reasoning (Liu et al., 2026b; Zhong et al., 2026). Adaptive early exit changes reasoning depth according to the needs of an individual decision (Lei & Shan, 2026). These approaches motivate efficient intermediate computation, but prediction, alignment, and confidence serve different purposes from checking whether a stage has actually completed. Our focus is the reuse and revision of stage-aligned segments across execution steps, with verification overhead included in the comparison of repair strategies.

In this setting, computation depends on both how much reasoning is generated at a decision and how much remains usable afterward. Stage verification addresses the latter by connecting retained latent context to observed progress and the prerequisites of subsequent operations.

Execution verification and recovery. RoboFailRing grounds failure diagnosis in retrieved information, AgentChord prepares recovery branches within a task graph, and HarnessWAM maintains task and scene context for execution verification and plan revision (Ying et al., 2026; Xu et al., 2026; Gu et al., 2026). They establish the importance of using execution evidence to preserve progress and adjust subsequent behavior. We study how such evidence selects a continuous latent suffix inside the VLA for reconstruction. LIBERO-RECOVER complements ordinary task-success evaluation with failure-state continuation at several recovery levels (Liu et al., 2026a). Our controlled experiments isolate stage supervision, semantic alignment, and repair scope; the published comparisons provide context rather than a controlled ranking against every monitoring or task-graph system.

3 METHOD

3.1 PROBLEM FORMULATION AND OVERVIEW

Given a language instruction g , the robot receives observation $o_t = (I_t, q_t)$, where I_t contains images and q_t is proprioception. A history summary M_t records verified stages, relevant object states, and execution outcomes. The model encodes this context and generates a plan comprising K stages. Each $Z_i \in \mathbb{R}^{m \times d}$ contains m continuous vectors with the backbone hidden dimension d . The action policy conditions on the active stage k and the remaining latent segments:

$$\begin{aligned} h_t &= E(g, o_t, M_t), & Z_i &= F_\theta(h_t, Z_{<i}), & i &= 1, \dots, K, \\ a_{t:t+H-1} &\sim \pi_\phi(\cdot \mid o_t, g, M_t, Z_{k:K}). \end{aligned} \quad (1)$$

Execution alternates between short action chunks and renewed observation. The robot executes the first h steps of a predicted H -step chunk before checking progress again. Supported stage completion advances k ; persistent deviation can update the remaining plan. Simulator states are used only for offline labels and independent evaluation; deployment uses images, language, and proprioception.

The stage index i and environment time t therefore serve different roles: a stage can require several action chunks, and its latent-vector count does not specify its execution duration. Training stage counts follow demonstration events, with masks for variable lengths. At inference, the model predicts subsequent stages and their termination position within the planning window. Reaching that position can prompt a rolling extension; it does not itself certify that the physical task is complete.

3.2 STAGE ALIGNMENT AND TRAINING

Semantic stage targets. Demonstration boundaries follow stable, task-relevant events, including grasp establishment, placement, spatial containment, and container opening. At boundary τ_i , the target combines object relations with subgoal-completion signals:

$$s_i^* = (r_{\tau_i}^*, c_{\tau_i}^*). \quad (2)$$

Relations describe physical changes; completion signals indicate whether those changes satisfy the instruction. For example, being inside a container is a relation, while being inside the instructed

Algorithm 1 Stage verification and latent-suffix repair (module interfaces)**Require:** Instruction g , observed history M_t , budget B ; trained modules and stage-condition checks

```

1: Generate  $Z_{1:K}$  and set  $k = 1$ ; enforce the remaining budget before every model call.
2: while the task is unfinished and the budget permits the next call do
3:   Observe  $o_t$ ; estimate  $\tilde{s}_t$ ; check candidate completion or persistent deviation.
4:   if required evidence is insufficient then
5:     Keep the stage unconfirmed; do not advance or certify the prefix from missing evidence.
6:   else if completion is supported by consecutive valid observations then
7:     Update verified history and advance  $k$ ; extend the planning window when needed.
8:   else if reliable persistent evidence confirms a repair trigger then
9:     Check still-required conditions of earlier stages and form  $\mathcal{A}_t$ .
10:    Set  $j = \min(\{k\} \cup \mathcal{A}_t)$ ; retain the prefix verified as unaffected.
11:    Regenerate  $Z'_{j:K'}$  from  $g, o_t, M_t, Z_{<j}$  and set  $k = j$ .
12:   end if
13:   If actions remain and budget permits, decode a chunk, execute  $h$  steps, and update costs.
14: end while

```

target container contributes to the relevant subgoal. This separates the meaning of a stage from its duration and ties its endpoint to the task. Masks $m_{i,d}$ exclude state components whose labels cannot be established reliably, rather than treating uncertainty as a negative example.

Predicted and observed states. A decoder maps each latent segment to its expected endpoint. A separate observation estimator predicts the current state from recent images and proprioception over window w :

$$\hat{s}_i = D_\psi(Z_i), \quad \tilde{s}_t = V_\eta(o_{t-w:t}, g). \quad (3)$$

Its training examples cover observations before and after subgoal completion. Thus the expected outcome is a target to be checked, while the verification evidence comes from execution observations. Shared visual features do not make the latent prediction itself evidence of completion. The temporal window supplies evidence about transitions such as grasp establishment and object release, whose interpretation can depend on more than a single image.

Stage supervision compares decoded endpoints with the masked demonstration targets:

$$\mathcal{L}_{\text{stage}} = \sum_{i=1}^K \frac{\sum_d m_{i,d} \ell_s(\hat{s}_{i,d}, s_{i,d}^*)}{\sum_d m_{i,d} + \epsilon}. \quad (4)$$

This objective constrains the latent segments used for action generation, rather than only training an auxiliary decoder. It is combined with action imitation and observation-state estimation:

$$\mathcal{L} = \mathcal{L}_{\text{act}} + \lambda_{\text{stage}} \mathcal{L}_{\text{stage}} + \lambda_{\text{obs}} \mathcal{L}_{\text{obs}}. \quad (5)$$

Continuous actions use L1 loss; discrete predicates use masked binary cross-entropy. Stage supervision constrains the latent representation, action imitation makes it useful for control, and observation supervision trains the evidence source used at execution. Variable stage counts are handled with masks. Here, “verifiable” means that stage expectations have observable state counterparts; it does not imply a formal correctness guarantee for the latent computation.

Each loss is normalized over its valid training targets before weighting, so padded entries and unavailable predicates do not contribute supervision. Stage targets train the decoder and the latent representation that feeds it, whereas observation targets supervise state estimation from the actual trajectory. The action objective retains the demonstrated control sequence as its target. The same objectives are used to train conditional suffix generation at sampled stage boundaries.

3.3 OBSERVATION-BASED STAGE VERIFICATION

Verification concentrates on candidate stage endings, persistent stagnation, and violations of conditions needed by later operations. For active stage k , it compares the predicted endpoint with the observed state:

$$e_{t,k} = \frac{\sum_d w_{t,k,d} \ell_{\text{ver}}(\hat{s}_{k,d}, \tilde{s}_{t,d})}{\sum_d w_{t,k,d} + \epsilon}. \quad (6)$$

Weights combine task relevance and observation confidence, emphasizing the objects and conditions involved in the stage. A mismatch while the robot is normally approaching the endpoint is expected and does not alone trigger repair. Completion instead requires reliable support for the relevant conditions and subgoal across consecutive observations. Reliable, persistent contradictions can trigger repair; insufficient evidence leaves the stage **unconfirmed**. In particular, a small discrepancy with zero or negligible observation weight is not a completion certificate. The regularizer ϵ only stabilizes the calculation. These outcomes keep missing evidence distinct from both successful completion and a confirmed execution failure.

The training mask and verification weights have complementary purposes. The former selects trustworthy target components during learning; the latter determines which current observations can support a stage decision. When a required relation cannot be established from the available views, later observations can provide additional evidence without prematurely advancing the stage. Conversely, repeated evidence that a required state transition failed supports a repair trigger. This use of execution context prevents an endpoint comparison from treating every intermediate motion as a failure.

3.4 DEPENDENCY-AWARE LOCAL REPAIR

Once a repair trigger is confirmed, the model identifies the earliest stage whose outcome must be reconsidered. Let $\mathcal{A}_t$ contain earlier stages whose established conditions are reliably contradicted and still required by the remaining task. The start position and repaired latent plan are

$$\begin{aligned} j_t &= \min(\{k\} \cup \mathcal{A}_t), \\ \mathcal{Z}^{\text{new}} &= (Z_{<j_t}, F_\theta(g, o_t, M_t, Z_{<j_t})). \end{aligned} \quad (7)$$

If the deviation is confined to the current operation, reconstruction begins at k . If an earlier prerequisite has become invalid, the update expands to that stage. The retained prefix is checked against conditions still needed by future execution, not against every historical relation. For example, releasing an object after placing it is normal progress and does not invalidate the completed grasp. By contrast, loss of a relation required for the next operation can require an earlier repair start.

Dependencies are expressed through stage outcomes that supply prerequisites for later operations. In the drawer example, a failed grasp leaves the opening stage reusable if the drawer remains accessible. If the opening condition has been lost while retrieval still requires it, the affected set reaches back to that prerequisite. Reconstruction then includes subsequent segments whose expected states depended on the invalid condition. This check gives local repair a variable scope, from the active stage to an earlier portion of the plan.

The prefix supplies valid task history, while the latest observation supplies the actual starting conditions for the regenerated suffix. Retaining a completed stage does not repeat its physical actions. The suffix may change length to reflect the remaining task and recovery needs. During training, split positions are sampled at demonstration stage boundaries; the aligned prefix, boundary observation, instruction, and history condition reconstruction of the remaining stages and actions. Additional recovery trajectories use the same construction only in the separately reported data augmentation. This trains conditional suffix generation without treating every execution deviation as invalidation of the entire plan.

3.5 BUDGETED CLOSED-LOOP EXECUTION

Initial planning, ordinary rolling extensions, verification, and triggered repair all consume inference resources. We account for them through non-overlapping operations:

$$C_{\text{total}} = C_{\text{enc}} + C_{\text{init}} + C_{\text{roll}} + C_{\text{ver}} + C_{\text{act}} + \sum_n C_{\text{repair}}^{(n)} \leq B. \quad (8)$$

The terms denote shared visual encoding, initial planning, rolling extensions, verification, action decoding, and triggered regeneration. Shared encoding is charged once, and downstream terms exclude that cost. Both successful and failed episodes use the same accounting. Remaining budget constrains subsequent model calls and repair length; planning can extend as execution advances. Algorithm 1 summarizes the control flow and module interfaces. This accounting enables the repair comparison to include the cost of deciding when and where to update the latent plan.

Within the remaining budget, repair prioritizes the current stage and its directly dependent operations; more distant planning extends as execution proceeds. The reconstruction horizon is therefore distinct from the full task horizon. A shorter regenerated sequence can reduce latent generation, but its total benefit also depends on verification and action computation. We evaluate the complete loop rather than using latent-vector count as a substitute for inference time.

4 EXPERIMENTS

4.1 BENCHMARKS AND DATA

We evaluate LIBERO-Spatial, Object, Goal, and Long (LIBERO-10), with ten tasks per suite (Liu et al., 2023); RoboTwin 2.0 under clean and randomized conditions (Chen et al., 2025); and LIBERO-RECOVER at four failure-recovery levels (Liu et al., 2026a). RoboTwin results distinguish the 50-task benchmark from Fast-ThinkAct’s ten-task subset. Recovery macro-averages use Spatial, Object, Goal, and LIBERO-100, rather than substituting LIBERO-Long for the last recovery suite.

Stage labels are obtained by replaying demonstrations and mapping stable object-relation and subgoal events to predicates. Unreliable labels are masked. Splits use complete trajectories; recovery samples are grouped by failure scenario and source trajectory. The default configuration uses demonstrations and their stage annotations without additional recovery trajectories or dedicated initial-frame input. Augmentations are tested independently.

4.2 MODEL AND CONTROLLED COMPARISONS

The base policy uses pretrained OpenVLA and follows OpenVLA-OFT’s continuous action chunks and L1 objective (Kim et al., 2024; 2025). Each stage contains four latent vectors. The recorded planning configuration allows up to eight stages per window and extends longer tasks as execution advances. Action chunks contain $H = 8$ steps, of which $h = 4$ execute before observing again. Fine-tuning is separate for each benchmark, with action normalization from its training data only. Recorded optimization settings are given in Appendix A.

Controlled ablations keep initialization, task splits, training trajectories, observation access, and execution intervals consistent. Configurations requiring retraining use three independent seeds; evaluation uses paired initial states and failure scenarios. Repair strategies reuse model weights, the observation estimator, and trigger rules wherever possible. The base-policy row in Table 4 is our implementation under these controls; published OpenVLA-OFT scores in Table 1 are a separate reference.

4.3 METRICS AND REPORTING

Task success is the mean of per-task success rates, judged by environment goals within the evaluation limits. Stage-verification F1 uses an independent diagnostic observation set and environment-derived labels. Recovery includes every predetermined failure instance: undetected failures, unsuccessful repairs, and budget exhaustion all count as failures.

For suite s and recovery level ℓ , let $n_{s\ell}$ and $u_{s\ell}$ denote tested and successful instances. With $S = 4$ suites, Table 3 and Tables 4–6 use, respectively,

$$R_\ell = \frac{1}{S} \sum_s \frac{u_{s\ell}}{n_{s\ell}}, \quad R_{\text{overall}} = \frac{1}{S} \sum_s \frac{\sum_\ell u_{s\ell}}{\sum_\ell n_{s\ell}}. \quad (9)$$

These differ from pooling all suites into one instance-level rate; averaging R_ℓ generally does not recover R_{overall} . Appendix B gives the remaining definitions. Success and verification results for our configurations are reported as measured means; the training protocol uses three seeds. Standard deviations and confidence intervals are not supplied in the tables, and we make no statistical-significance claims. Timing reports mean complete-episode inference time, p95 decision latency, and mean re-generated vectors. The reported efficiency comparison is at the evaluated setting, without a multi-budget curve claim.

4.4 EXPERIMENTAL RESULTS AND COMPARISONS

Tables 1–3 retain published baselines under their original settings and report our measured means. Differences in data, inputs, and budgets limit causal interpretations of cross-paper rankings. Our component effects are examined in Section 4.5.

Table 1: LIBERO task success (%). Published references and our measurements.

Method	Source	Spatial	Object	Goal	Long	Average
Diffusion Policy	A	78.3	92.5	68.3	50.5	72.4
Octo	A	78.9	85.7	84.6	51.1	75.1
DiT Policy	A	84.2	96.3	85.4	63.8	82.4
OpenVLA	A	84.7	88.4	79.2	53.7	76.5
π_0 -FAST	A	96.4	96.8	88.6	60.2	85.5
π_0	A	96.8	98.8	95.8	85.2	94.2
OpenVLA-OFT	A	97.6	98.4	97.9	94.5	97.1
CoT-VLA	B	87.5	91.6	87.6	69.0	83.9
ThinkAct	B	88.3	91.4	87.1	70.9	84.4
MolmoAct	B	87.0	95.4	87.6	77.2	86.8
Fast-ThinkAct	B	92.0	97.2	90.2	79.4	89.7
LaRA-VLA	C	96.4	99.8	98.6	96.6	97.9
Ours	This work	98.0	98.8	98.4	96.8	98.0

A: Kim et al. (2025); B: Huang et al. (2026); C: Bai et al. (2026). Published averages retain their source definitions; our average equally weights the four suites. Cross-paper settings differ.

LIBERO. Our overall and Long success rates are 98.0% and 96.8%, compared with LaRA-VLA’s published 97.9% and 96.6%, and OpenVLA-OFT’s 97.1% and 94.5%. The 0.1-point overall difference from LaRA-VLA does not establish a significant advantage. Our Object and Goal scores, 98.8% and 98.4%, are below LaRA-VLA’s 99.8% and 98.6%. These results describe strong aggregate completion without uniform superiority across suites.

RoboTwin 2.0. On 50 tasks, our Easy/Hard success is 58.0%/23.0%, exceeding the highest listed published values by 2.8/6.7 points. On the ten-task subset it is 68.0%/31.0%, or 2.3/4.6 points above Fast-ThinkAct. The subsets have different task coverage. The large clean-to-randomized drop remains a limitation; DP3’s point-cloud input also differs from image-based policies, so rankings do not isolate reasoning quality.

Failure recovery. Our L1–L4 macro-averages are 52.0%, 32.0%, 20.0%, and 9.0%. L4 exceeds the highest listed published reference, 5.6%, by 3.4 points, while remaining low in absolute terms. Recovery depends on action capability, observation quality, and localization as well as verification; detecting a discrepancy alone does not supply the actions needed to recover.

Inference cost. Table 5 includes encoding, verification, action decoding, and every planning or repair call. Local repair takes 14.9 seconds per episode versus 18.6 for full replanning, a 19.9% reduction, with recovery of 26.0% versus 26.4%. This is a measured trade-off under the evaluated

Table 2a: RoboTwin 2.0: 50 tasks (%).

Method	Easy	Hard
DP	28.0	0.6
ACT	29.7	1.7
RDT	34.5	13.7
π_0	46.4	16.3
DP3	55.2	5.0
Ours	58.0	23.0

Published baselines: Chen et al. (2025). The protocol uses 50 clean demonstrations per task and 100 tests per condition.

Table 2b: RoboTwin 2.0: 10-task subset (%).

Method	Easy	Hard
DP	43.1	0.6
ACT	45.5	3.5
π_0	52.9	16.3
RDT	56.4	22.8
ThinkAct	62.4	24.7
Fast-ThinkAct	65.7	26.4
Ours	68.0	31.0

Published baselines: Huang et al. (2026). Task coverage differs from Table 2a; the two averages are not directly comparable.

Table 3: LIBERO-RECOVER: macro-average success at each recovery level (%).

Method	L1	L2	L3	L4
π_0	46.1	24.4	13.5	4.6
π_0 -FAST	40.0	22.9	16.1	3.7
GR00T-N1.5	30.7	25.3	15.2	1.5
OpenVLA-OFT	47.7	26.6	14.2	5.6
Wan2-Policy	17.8	16.2	12.5	3.9
Cosmos-Predict2-Policy	15.5	14.5	9.9	0.0
Ours	52.0	32.0	20.0	9.0

Published values: Liu et al. (2026a). Each level equally weights Spatial, Object, Goal, and LIBERO-100. L1-L4: action retry, action adaptation, object-state recovery, and environment-state recovery.

settings, rather than evidence of better recovery at every budget. Published Fast-ThinkAct latency comparisons are discussed separately in Appendix C.

4.5 ABLATIONS AND ANALYSIS

Stage alignment and supervision. Auxiliary state supervision uses the same state labels without constructing a staged latent chain. The no-alignment variant retains the chain, decoder, and observation estimator, but blocks stage-state supervision gradients into latent representations and their upstream encoder; the decoder remains trained. The repair-disabled variant retains observation inputs and closed-loop actions while prohibiting deviation-triggered latent reconstruction.

Table 4: Measured component ablations. Success, recovery, and F1 are in %.

Configuration	LIBERO Avg.	LIBERO Long	Recovery	Stage F1
Base policy: no latent reasoning	96.8	94.2	20.5	N/A
Auxiliary state supervision only	97.1	94.8	21.3	N/A
Without latent stage alignment	97.2	95.0	22.1	81.6
Execution-feedback repair disabled	97.5	95.8	22.8	92.4
Complete method	98.0	96.8	26.0	92.4

All rows are measured means in this work. Recovery equally weights suite-level rates, as in Table 6. N/A: no latent-stage verification interface. The base policy is our controlled implementation; Table 1 reports published OpenVLA-OFT results.

The complete method improves average success, Long success, and recovery over the base policy by 1.2, 2.6, and 5.5 points. Auxiliary supervision alone gains 0.3, 0.6, and 0.8 points; the complete model remains 0.9, 2.0, and 4.7 points higher. Removing alignment reduces verification F1 from 92.4% to 81.6% and recovery from 26.0% to 22.1%. Disabling repair reduces recovery to 22.8%. Its identical 92.4% F1 reflects shared verifier weights evaluated on the same fixed diagnostic observations, separating static verification accuracy from its use during execution.

Repair scope. Full replanning receives the same observation, instruction, and valid history and regenerates all latent representations of the remaining plan; it does not repeat already-completed physical actions. Current-stage-only reconstruction always begins at the active stage and cannot revisit earlier prerequisites. Dependency-aware repair checks those prerequisites before choosing the suffix.

Table 5: Measured repair strategies: recovery and complete-episode inference cost.

Repair strategy	Recovery (%)	Time (s/episode)	p95 latency (ms)	Vectors (/episode)
Repair disabled	22.8	11.8	180	0
Full replanning	26.4	18.6	820	62.4
Current-stage-only suffix reconstruction	24.6	14.2	360	23.6
Dependency-aware local repair	26.0	14.9	430	31.2

Same recovery instances, step limits, and task-level budget. Time includes all model calls and both successful and failed episodes; shared encoding is counted once. Regenerated vectors count triggered repairs only; ordinary rolling extensions are excluded from that count. Means are reported except for p95 latency.

Relative to full replanning, local repair lowers cumulative time by 19.9%, p95 decision latency from 820 to 430 ms (47.6%), and regenerated vectors from 62.4 to 31.2 (50.0%), with recovery lower by 0.4 points. Current-stage-only repair is 0.7 seconds faster than dependency-aware repair but 1.4 points lower in recovery. Enabling complete repair over disabling it increases recovery by 3.2 points at 3.1 additional seconds. These measurements support a trade-off across repair scopes; they do not separate prefix-reuse savings from all effects of episode termination.

Recovery data and observation history. Each augmentation in Table 6 is applied independently to the original configuration. Adding recovery trajectories changes training data; adding the initial frame keeps trajectories fixed and changes available visual input. Its encoding contributes to inference cost.

Table 6: Recovery data and initial-frame input: suite macro-average success (%).

Method	Original	Recovery data	Initial frame
OpenVLA-OFT	20.8	25.4	26.8
GR00T-N1.5	17.8	21.2	18.9
Ours	26.0	31.0	28.0

Published baselines: Liu et al. (2026a). Each augmentation is independently compared with the original configuration. Pool recovery instances within each suite, then equally average the four suites; averaging the level columns of Table 3 is a different statistic.

Recovery increases from 26.0% to 31.0% with recovery data and to 28.0% with initial-frame input. These are separate gains of 5.0 and 2.0 points and cannot be added to predict their joint effect. Equation 9 explains why the 26.0% overall recovery rate need not equal the arithmetic mean of Table 3’s four level scores.

5 CONCLUSION

We presented a VLA framework that connects stage-supervised latent reasoning, observation-based verification, and selective suffix repair. The method reaches 98.0% average LIBERO success, 96.8% on Long tasks, and a 26.0% recovery macro-average. Relative to full replanning, local repair reduces complete-episode inference time by 19.9%, with a 0.4-point recovery difference. Controlled ablations support the roles of stage alignment and execution feedback under the evaluated settings. These results suggest that preserving valid progress can reduce repeated reasoning while maintaining task continuation. The evidence remains limited by weak environment-state recovery, simulation-derived annotations, and evaluation at the reported budget setting. Future work should examine real-robot transfer, weaker stage supervision, and recovery-cost behavior across budgets.

AI USE STATEMENT

OpenAI Codex assisted with manuscript organization, drafting and Chinese-English translation, literature lookup, consistency checks, and LaTeX formatting. The reported experimental values were supplied in the authors’ records. This assistance did not include running the robot experiments or inspecting the experiment implementation. Responsibility for the scientific claims and final manuscript rests with the authors.

REPRODUCIBILITY STATEMENT

Section 3 and Algorithm 1 specify the reasoning, verification, and repair interfaces. Section 4.3 and Appendix B define result aggregation. Appendix A records the training and annotation settings described for this study. Tables 4 and 5 identify controlled comparisons; published reference scores retain the scope of their original studies.

REFERENCES

- Shuanghao Bai, Jing Lyu, Wanqi Zhou, Zhe Li, Dakai Wang, Lei Xing, Xiaoguang Zhao, Pengwei Wang, Zhongyuan Wang, Cheng Chi, Badong Chen, and Shanghang Zhang. Latent Reasoning VLA: Latent Thinking and Prediction for Vision-Language-Action Models. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.01166v2>.
- Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan-ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. *arXiv preprint arXiv:2506.18088v2*, 2025. URL <https://arxiv.org/abs/2506.18088v2>.
- Zhaopeng Gu, Bingke Zhu, Tianxi Lin, Guibo Zhu, Yingying Chen, Kai Wang, Tingyu Yuan, Chaoyang Zhao, Zhaowen Li, Peng Su, and Jinqiao Wang. HarnessWAM: Bridging Prediction and Deliberation in World Action Models. *arXiv preprint arXiv:2608.09516*, 2026. URL <https://arxiv.org/abs/2608.09516>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Chi-Pin Huang, Yunze Man, Zhiding Yu, Min-Hung Chen, Jan Kautz, Yu-Chiang Frank Wang, and Fu-En Yang. Fast-ThinkAct: Efficient Vision-Language-Action Reasoning via Verbalizable Latent Planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. URL <https://arxiv.org/abs/2601.09708v2>.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Open-VLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024. URL <https://arxiv.org/abs/2406.09246>.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. In *Robotics: Science and Systems*, 2025. URL <https://arxiv.org/abs/2502.19645v2>.
- Dianqiao Lei and Lianlei Shan. Think Less, Act Early: Reinforced Latent Reasoning with Early Exit in Vision-Language-Action Models. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2606.15099>.
- Xiang Li, Ya-Li Li, Yuan Wang, Huaqiang Wang, and Shengjin Wang. TCoT: Trajectory Chain-of-Thoughts for Robotic Manipulation with Failure Recovery in Vision-Language-Action Model.

- Proceedings of the AAAI Conference on Artificial Intelligence*, 40(8):6486–6494, 2026. doi: 10.1609/aaai.v40i8.37577. URL <https://ojs.aaai.org/index.php/AAAI/article/view/37577>.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.03310>.
- Lin Liu, Zhicheng Bao, Lu Zhang, Ziyang Song, Wu Yang, Yuzheng Zhuang, Shuai Tao, Wulong Liu, Caiyan Jia, and Huchuan Lu. LIBERO-RECOVER: Beyond Task Success Towards Failure Recovery in Robotic Manipulation Models. *arXiv preprint arXiv:2609.05178v2*, 2026a. URL <https://arxiv.org/abs/2609.05178v2>.
- Zhuoyang Liu, Jiaming Liu, Hao Chen, Jiale Yu, Ziyu Guo, Chengkai Hou, Chenyang Gu, Xiangju Mi, Renrui Zhang, Kun Wu, Zhengping Che, Jian Tang, Pheng-Ann Heng, and Shanghang Zhang. LaST₀: Latent Spatio-Temporal Chain-of-Thought for Robotic Vision-Language-Action Model. *arXiv preprint arXiv:2601.05248v4*, 2026b. URL <https://arxiv.org/abs/2601.05248v4>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Huajie Tan, Peterson Co, Yijie Xu, Shanyu Rong, Yuheng Ji, Cheng Chi, Xiansheng Chen, Qiongyu Zhang, Zhongxia Zhao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Action-Sketcher: From Reasoning to Action via Visual Sketches for Long-Horizon Robotic Manipulation. *arXiv preprint arXiv:2601.01618v1*, 2026. URL <https://arxiv.org/abs/2601.01618v1>.
- Sheng Xu, Ruixing Jin, Huayi Zhou, Bo Yue, Guanren Qiao, Yunxin Tai, Yueci Deng, Kui Jia, and Guiliang Liu. From Reaction to Anticipation: Proactive Failure Recovery through Agentic Task Graph for Robotic Manipulation. In *Robotics: Science and Systems*, 2026. URL <https://arxiv.org/abs/2605.11951>.
- Chenduo Ying, Linkang Du, Yuanchao Shu, and Peng Cheng. RoboFailRing: Retrieval-Augmented and Language Grounding Failure Detection for VLM-enabled Robotic Manipulation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13188–13202, 2026. doi: 10.18653/v1/2026.acl-long.602. URL <https://aclanthology.org/2026.acl-long.602/>.
- Yifu Yuan, Haiqin Cui, Yaoting Huang, Yibin Chen, Fei Ni, Zibin Dong, Pengyi Li, Yan Zheng, Hongyao Tang, and Jianye Hao. Embodied-R1: Reinforced Embodied Reasoning for General Robotic Manipulation. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2508.13998v2>.
- Zhide Zhong, Junfeng Li, Junjie He, Haodong Yan, Xin Gong, Guanyi Zhao, Yingjie Cai, Jiantao Gao, Xu Yan, Bingbing Liu, Yingcong Chen, Liuqing Yang, and Haoang Li. DualCoT-VLA: Visual-Linguistic Chain of Thought via Parallel Reasoning for Vision-Language-Action Models. *arXiv preprint arXiv:2603.22280*, 2026. URL <https://arxiv.org/abs/2603.22280>.

A TRAINING, ANNOTATION, AND VERIFICATION DETAILS

The policy is fine-tuned separately on each benchmark. Single-arm and bimanual action dimensions follow their respective interfaces; action-normalization statistics use only the corresponding training set. The recorded optimizer is AdamW with initial learning rate 10^{-4} , effective batch size 64, weight decay 0.01, and BF16 training (Loshchilov & Hutter, 2019). LoRA adapters have rank 32; latent, state-decoding, observation-estimation, and action heads are trained (Hu et al., 2022). The initial stage and observation loss weights are both 1, with checkpoint and weight selection performed on validation data. These are the settings documented for model training; the efficiency comparison uses identical hardware, precision, batch size, warm-up, and synchronization across strategies.

Stage boundaries follow stable changes in task-relevant predicates: support-to-grasp transitions, object release or placement, spatial containment, and container opening or closing. Targets combine physical relations with whether those relations satisfy the instruction’s subgoal. Uncertain annotations are masked rather than converted into negative labels. Demonstration splits respect complete trajectories; recovery variants are grouped by failure scenario and source trajectory to avoid near-duplicate variants crossing splits. Simulator access is restricted to offline annotation and independent scoring.

Each stage contains four latent vectors with the backbone hidden dimension. Masks handle varying training-stage counts. Inference predicts subsequent stages and termination within a planning window, with a documented initial limit of eight stages per window. Longer tasks extend their plan as the window is completed. The action head conditions on the current observation, instruction, valid history, and subsequent latent segments.

Observation-based estimation uses recent images and proprioception. Candidate completion, persistent stagnation, and violations of required conditions initiate checks. The recorded initial completion configuration requires three consecutive valid confirmations; thresholds and observation settings are selected on validation data. Verification has three outcomes: supported completion, reliable persistent contradiction, and insufficient evidence. A low weighted discrepancy is not a sufficient completion condition if relevant predicates remain unobserved. The denominator regularizer in Equation 6 provides numerical stability only.

The dependency check concerns conditions still needed by the remaining task. If a placed object has legitimately been released, the completed grasp need not be repeated. If a still-required relation from an earlier stage has been invalidated, the update starts from that earlier stage. Equation 7 provides notation for this scope selection, while Algorithm 1 describes the module interfaces.

Suffix-generation training samples split positions at demonstration stage boundaries. Inputs are the aligned prefix, the corresponding observed state, instruction, and task history; supervision uses the remaining stages and actions. In the recovery-data augmentation, actual recovery trajectories supply the suffix and action targets from failure states. The original configuration in Tables 4–6 excludes these additional trajectories.

B METRIC DEFINITIONS AND ACCOUNTING

For M tasks, let N_i be the number of test episodes and y_{in} indicate that task i succeeds within its limits. Task success is

$$\text{SR} = \frac{1}{M} \sum_{i=1}^M \frac{1}{N_i} \sum_{n=1}^{N_i} y_{in}. \quad (10)$$

The LIBERO average equally weights the Spatial, Object, Goal, and Long suite scores. Each suite has ten tasks. RoboTwin’s 50-task and ten-task averages refer to their respective task sets and are not combined into a single ranking. The original 50-task protocol uses 50 clean demonstrations per task and 100 tests under each clean/randomized condition. The listed ten-task references retain the Fast-ThinkAct protocol.

Stage-completion verification uses environment-derived reference labels on fixed observations before and after completion. True positives confirm completed stages, false positives incorrectly confirm unfinished stages, and false negatives miss completed stages. The reported diagnostic is

$$F1_{\text{verify}} = \frac{2TP}{2TP + FP + FN}. \quad (11)$$

For N_f predefined recovery instances and success indicators z_n , the pooled instance-level recovery rate is

$$RSR_{\text{pooled}} = \frac{1}{N_f} \sum_{n=1}^{N_f} z_n. \quad (12)$$

This pooled definition must be distinguished from the suite macro-averages in Equation 9. In Table 3, each level equally weights four suites. For the overall score in Tables 4–6, instances are first pooled within each suite, then the four suite scores are averaged. If the level proportions vary across suites, equally averaging the four level columns produces a different result. Thus 52.0%, 32.0%, 20.0%, and 9.0% in Table 3 need not average to the 26.0% overall score. Neither aggregation discards undetected failures or budget-exhausted episodes.

Cumulative inference time sums all model operations in Equation 8 during each test episode. Environment dynamics are distinguished from model computation. Visual encoding shared by action and verification is timed once. Latent-vector regeneration counts only deviation-triggered reconstruction; ordinary rolling extensions are accounted for in time but excluded from that auxiliary vector count. Mean episode time covers successful and failed episodes. Decision latency measures observation-to-executable-action delay; Table 5 reports its 95th percentile. Hardware, numerical precision, batching, warm-up, and synchronization are matched across repair strategies.

These aggregate timings do not independently identify every source of savings. Full replanning achieves the highest recovery score in Table 5; local repair regenerates fewer vectors but may also change the distribution of episode termination times. The evidence supports the reported setting’s recovery-cost trade-off. A general success-versus-budget frontier or equal-success comparison would require separate measurements and is not claimed here. Likewise, measured means without uncertainty estimates do not establish the significance of small differences.

C PUBLISHED CONTEXT AND SCOPE OF THE EVIDENCE

The LIBERO reference rows retain the source grouping in Table 1: A is OpenVLA-OFT, B is Fast-ThinkAct, and C is LaRA-VLA. Columns are reordered consistently as Spatial, Object, Goal, and Long; published averages remain as reported. Cross-paper comparison can reflect different training data, observations, or budgets. It is therefore descriptive context for the controlled comparisons in Tables 4 and 5.

Fast-ThinkAct reports inference latencies of 805, 5674, and 7513 ms for Fast-ThinkAct-3B, ThinkAct-3B, and ThinkAct-7B, respectively (Huang et al., 2026). The corresponding reductions of approximately 85.8% and 89.3% describe those original model comparisons. They are separate from our measured 19.9% complete-episode reduction between local repair and full replanning, and use different timing settings.

Published ablations also illustrate why supervision should be controlled. Removing Fast-ThinkAct’s verbalization preference loss changes LIBERO success from 89.7% to 88.6%; removing that loss and visual distillation yields 86.3% (Huang et al., 2026). LaRA-VLA reports 55.2% versus 68.8% on SimplerEnv for its no-CoT configuration and joint latent text-vision supervision (Bai et al., 2026). These values do not test our modules. Our auxiliary-state-only baseline and no-alignment variant instead hold the stated training setup fixed while testing additional supervision and the stage-semantic constraint.

The complete method’s gains over auxiliary supervision involve both a staged latent representation and execution-feedback repair. The no-alignment comparison isolates the contribution of gradient-based stage-semantic alignment while keeping the verification decoder trained. Disabling repair tests

the use of verifier outputs for execution, with static diagnostic inputs and verifier weights unchanged. These comparisons support the roles of the components, but do not establish superiority over a separately implemented explicit task graph with otherwise identical supervision and feedback.

Full replanning can be useful when prerequisites fail broadly or historical verification is wrong. Current-stage-only reconstruction can be cheaper when failures remain local, but cannot revisit earlier conditions. Observation errors under domain randomization can create false triggers, and environment-state recovery requires action capabilities beyond discrepancy detection. The low 9.0% L4 score and the 58.0%-to-23.0% clean/randomized gap delimit the current evidence. Evaluation is in simulation; real-robot generalization is not established by these results.